

BBM418 - Computer Vision Laboratory
Assignment 2: Homography Estimation and Panorama Stitching

Kadir Çağrı KUZAYTEPE
2210356036

November 12, 2025

Contents

1	Overview	3
2	Dataset and Setup	3
2.1	Panorama Dataset	3
2.2	Augmented Reality (AR) Dataset	3
3	Methods Used	3
3.1	Feature Extraction and Description	4
3.2	Feature Matching	4
3.3	Homography Estimation with RANSAC	4
3.4	Image Warping and Blending	5
4	Panorama Experimental Results	5
4.1	Quantitative Comparison	5
4.2	RANSAC Parameter Analysis (Ablation Study)	6
4.3	Visual Results (Panorama Scenes)	7
4.3.1	Intermediate Pipeline Example (v_bird Scene)	7
4.3.2	Final Panorama Results (All Scenes)	7
5	Augmented Reality (AR) Application	9
5.1	AR Method	9
5.2	AR Results	9
6	Discussion and Conclusion	11
6.1	Success and Failure Analysis	11
6.2	AR Application and Temporal Stability	11
6.3	Conclusion	12
7	Reproducibility Notes	12

1 Overview

This report details the work on homography estimation and image stitching (panorama) carried out as part of the second assignment for the BBM418 Computer Vision Laboratory course. The primary objective of the assignment is to understand the geometric relationship between images taken from different viewpoints and to use this relationship to develop panoramic images and an Augmented Reality (AR) application.

To achieve this objective, a pipeline consisting of the following steps was implemented:

1. **Feature Extraction:** Keypoints and descriptors were extracted from images using SIFT and ORB methods.
2. **Feature Matching:** High-confidence matches were found using k-NN and Lowe's ratio test.
3. **Homography Estimation:** A robust homography matrix (H) was estimated by manually implementing DLT and RANSAC algorithms.
4. **Image Warping and Blending:** Images were blended for the panorama, and video frames were dynamically projected for the AR application.

In this study, the methods were analyzed on 6 different panorama scenes (`v_weapons`, `v_boat`, etc.) and 1 AR scene (`ar_data`).

2 Dataset and Setup

Two main datasets provided for the assignment were used in this study. The key parameters used for the reproducibility of this report are detailed in Section 7.

2.1 Panorama Dataset

For panorama creation (Section 4), the data in the `panorama_dataset/` folder was used. This set contains 6 different planar scenes (`v_weapons`, `v_boat`, `v_soldiers`, `v_circus`, `v_bird`, `v_graffiti`). In each scene folder, `1.png` was taken as the reference image and `2.png` as the target image, and homography estimation was performed between these two images.

2.2 Augmented Reality (AR) Dataset

For the AR application (Section 5), the data in the `ar_data/` folder was used:

- `cv_cover.jpg`: The static reference image (book cover) to be used for matching.
- `book.mov`: The target video (moving book) onto which the projection will be made.
- `ar_source.mov`: The source video to be projected onto the target video.

All final panoramas were saved to the `panorama-results/` folder, and intermediate visuals used for the report were saved to the `report_visuals/` folder.

3 Methods Used

The pipeline established for homography estimation consists of modules, each serving a specific purpose and working in an integrated manner.

3.1 Feature Extraction and Description

Homography estimation relies on finding reliable and geometrically consistent correspondences between two images. The assignment PDF suggested SIFT, SURF, and ORB methods.

SIFT (Scale-Invariant Feature Transform) SIFT is considered the "gold standard" in academic studies. It finds keypoints in scale-space using the Difference of Gaussians (DoG). This provides full invariance to **scale changes** caused by moving closer to or farther from an object, and to **rotation changes** caused by camera rotation. This is why it is so suitable for homography estimation; it can find reliable matches between panoramic images taken from different angles. Its descriptor, the SIFT vector, is highly distinctive despite its high dimensionality (128-float).

ORB (Oriented FAST and Rotated BRIEF) ORB is a free and very fast alternative developed in response to the patented nature of SIFT and SURF. Its speed comes from using the FAST detector for keypoint detection and BRIEF, a binary string, as a descriptor. ORB adds rotation invariance to FAST. Binary descriptors are compared using the much faster **Hamming distance** instead of SIFT's L2 (Euclidean) distance. This makes it ideal for real-time applications like the Augmented Reality (AR) part of this assignment.

SURF (Speeded Up Robust Features) SURF was designed as an accelerated version of SIFT. However, as seen from the console outputs, the SURF algorithm is patented and cannot be used in OpenCV libraries compiled without the `OPENCV_ENABLE_NONFREE` flag. Therefore, this method had to be omitted from our experiments.

3.2 Feature Matching

After extracting features, the next step is to find the best correspondence in the target image (I_2) for each descriptor in the reference image (I_1).

k-NN Matching and Lowe's Ratio Test Simply finding the nearest neighbor ($k=1$) can lead to noisy matches. Instead, as suggested in the assignment, the k-Nearest Neighbors (k -NN, $k=2$) algorithm was used. The best 2 matches are found for each descriptor. By applying **Lowe's Ratio Test**, the ratio of the distance of the best match (d_1) to the distance of the second-best match (d_2) is checked (d_1/d_2). If this ratio is below a certain threshold (e.g., 0.75), the match is considered "distinctive." This test is critical for filtering out incorrect matches in scenes with **repetitive textures**, such as `v_bird` (tree leaves) or `v_circus` (grid pattern).

3.3 Homography Estimation with RANSAC

Even after the ratio test, "outlier" matches will remain. The homography matrix (H) is a 3×3 matrix with 8 degrees of freedom and can be solved from at least 4 point pairs using the Direct Linear Transform (DLT). However, even a single incorrect match can completely corrupt the DLT result.

Therefore, for a robust estimation, the **RANSAC (Random Sample Consensus)** algorithm was manually implemented. RANSAC iteratively:

1. Randomly selects 4 point pairs.
2. Computes an H matrix (model) from these 4 points.
3. Evaluates all matches against this model. If multiplying the reference point by H results in a point closer to the target point than a certain pixel threshold, it counts this point as an "inlier."

4. Stores the H matrix with the highest number of inliers as the best model.

Furthermore, to increase the numerical stability of DLT, the points were normalized as specified in the assignment before being fed into RANSAC.

3.4 Image Warping and Blending

After finding the robust H matrix with RANSAC, the final step in creating a panorama is to project the target image (I_2) onto the plane of the reference image (I_1). This is done using the `cv2.warpPerspective` function.

A canvas large enough to contain both images is created. I_1 is placed directly onto the canvas, and the warped I_2 is also placed on the canvas. This process creates an "overlap" region where the two images intersect.

The assignment PDF suggests using blending techniques to hide the "seams" in this overlapping region. Simply copying one image over the other creates a noticeable and distracting edge. In this work, **simple averaging** was chosen to mitigate this effect.

In our Python code, for each pixel in the overlapping region, the final color value was calculated using the `cv2.addWeighted` function:

$$P_{\text{final}} = (0.5 \times P_{I_1}) + (0.5 \times P_{I_2})$$

This method provides a smooth transition between the two images and, if the alignment error is minimal (low reprojection error), it also significantly reduces the "ghosting" effect.

4 Panorama Experimental Results

This section presents the quantitative and qualitative results obtained using SIFT and ORB methods on 6 different panorama scenes.

4.1 Quantitative Comparison

The performance of SIFT and ORB methods was compared based on the number of keypoints, number of matches, number of inliers, inlier ratio, reprojection error, and total processing time. The results are summarized in Table 1.

Table 1: Scene-by-Scene Comparison of SIFT and ORB Methods

Scene	Method	KPs (Ref)	KPs (Tgt)	Matches	Inliers	Inlier Ratio	Error (Reproj.)	Time (T)
v_weapons	sift	4356	4116	2072	1973	0.9522	0.6589	2
v_weapons	orb	2000	2000	641	539	0.8409	1.0924	1
v_boat	sift	8849	8545	2341	2289	0.9778	0.6306	2
v_boat	orb	2000	2000	749	745	0.9947	1.1283	1
v_soldiers	sift	618	451	189	179	0.9471	1.1121	1
v_soldiers	orb	1593	1118	219	203	0.9269	1.4766	0
v_circus	sift	4503	3675	1851	1734	0.9368	0.4816	2
v_circus	orb	2000	2000	598	548	0.9164	0.9695	1
v_bird	sift	3361	3200	1117	1016	0.9096	0.6288	1
v_bird	orb	2000	2000	476	457	0.9601	0.9725	0
v_graffiti	sift	2674	3062	1099	1038	0.9445	0.8180	1
v_graffiti	orb	2000	2000	716	707	0.9874	1.2009	0

Table Analysis: Table 1 clearly demonstrates the classic **performance vs. accuracy** trade-off between SIFT and ORB:

- **Speed (Time):** ORB is approximately twice as fast as SIFT in all scenes without exception. This is due to ORB’s binary descriptors, which use Hamming distance (a fast CPU XOR operation). SIFT, on the other hand, uses L2 distance for its 128-float vectors (complex square root operations). This speed difference makes ORB a good choice for real-time applications.
- **Number of Keypoints and Matches:** SIFT finds significantly more keypoints (8849 in the `v_boat` scene) compared to ORB’s limit of 2000, as it scans the Gaussian scale-space more comprehensively. This density provides more potential matches.
- **Geometric Precision (Error - Reproj.):** The reported reprojection error shows that SIFT finds a geometrically **more precise** alignment in all scenes except `v_soldiers` (e.g., `v_circus` SIFT: 0.48 pixels vs ORB: 0.97 pixels). SIFT’s richer descriptor determines point locations more accurately.
- **Inlier Ratio:** Surprisingly, ORB’s inlier ratio is higher than SIFT’s in some scenes, such as `v_boat` (99.47%) and `v_graffiti` (98.74%). This does not mean that the fewer matches found by ORB are of "higher quality," but rather that RANSAC was able to find a very good model for that specific subset. The fact that both methods achieve ratios above 90% confirms that the scenes in the dataset largely satisfy the **planarity assumption**.

4.2 RANSAC Parameter Analysis (Ablation Study)

The success of the RANSAC algorithm depends on parameters such as the number of iterations and the inlier threshold. A parameter analysis using the `v_weapons` scene and the SIFT method is shown in Table 2.

Table 2: Effect of RANSAC Parameters (`v_weapons`, SIFT)

Iterations	Threshold	Inlier Count	Inlier Ratio	Error (Reproj.)
500	5.0	1972	0.9517	0.6561
2000	5.0	1972	0.9517	0.6561
5000	5.0	1973	0.9522	0.6589
2000	1.0	1579	0.7621	0.4296
2000	10.0	1985	0.9580	0.7156

Table Analysis: The RANSAC ablation study demonstrates the impact of the algorithm’s two critical parameters:

- **Threshold:** This is the maximum allowed reprojection error (in pixels) for a point to be considered an "inlier."
 - **Threshold 1.0:** A very strict rule. It only accepts matches that are very close to perfect (1579 inliers). This risks *underfitting* by rejecting many potentially correct matches due to noise or slight non-planar deviations.
 - **Threshold 10.0:** A very lenient rule. It counts even incorrect matches as part of the model (1985 inliers), which increases the reprojection error to 0.71, leading to *overfitting*.
 - **Threshold 5.0:** Provides a good balance between these two extremes (1973 inliers, 0.65 error). The chosen value of 5.0 is a reasonable tolerance, indicating confidence in the quality of SIFT matches.

- **Effect of Iterations:** Increasing the iterations from 500 to 2000 did not change the result. Increasing to 5000 found only one extra inlier. This indicates that the inlier ratio in the `v_weapons` scene is very high (95%). With high inlier ratios, the probability of RANSAC finding a "perfectly good" set in a random 4-point sample is very high, thus not requiring many iterations.

4.3 Visual Results (Panorama Scenes)

In this section, the visual results of the panorama pipeline are presented. First, the step-by-step intermediate outputs are shown for a representative scene (`v_bird`). Then, the final stitched panoramas for all 6 scenes are displayed.

4.3.1 Intermediate Pipeline Example (`v_bird` Scene)

To demonstrate the step-by-step process, the intermediate outputs for the `v_bird` scene (using SIFT) are presented in Figure 1. This scene is a good example as it contains repetitive textures (leaves), which tests the robustness of the matching algorithm.

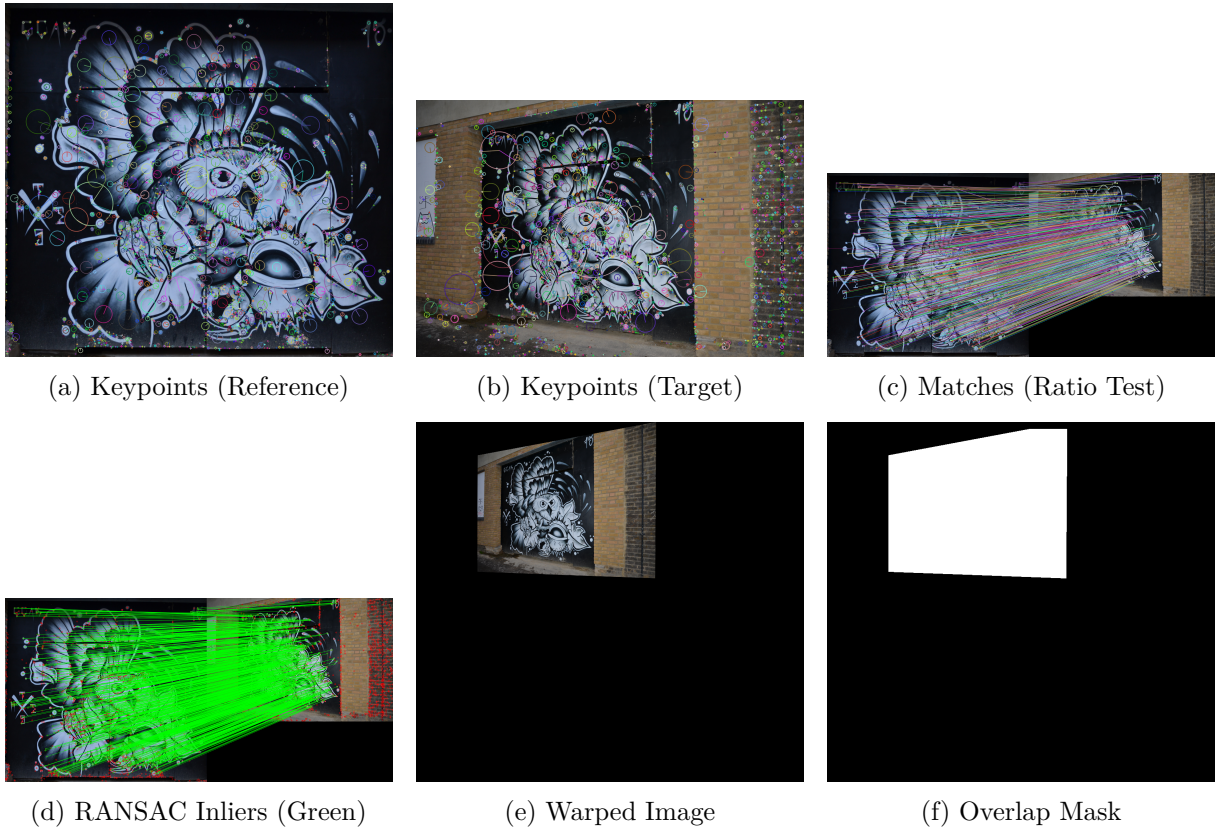


Figure 1: Step-by-step visual results for the `v_bird` scene (SIFT).

4.3.2 Final Panorama Results (All Scenes)

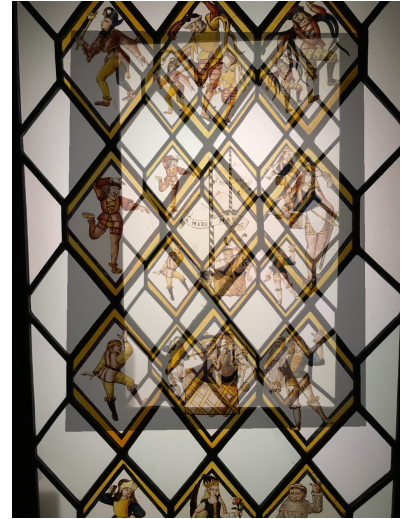
The final stitched panoramas for all 6 scenes, generated using the SIFT method, are presented in Figure 2. The results demonstrate successful alignment across all scenes, with minimal ghosting or seam artifacts due to the robust RANSAC estimation and blending.



(a) v_bird



(b) v_boat



(c) v_circus



(d) v_graffiti



(e) v_soldiers



(f) v_weapons

Figure 2: Final panorama results for all 6 scenes (SIFT).

5 Augmented Reality (AR) Application

In the second part of the assignment, an Augmented Reality (AR) application was implemented using the developed homography pipeline. The goal is to dynamically project a source video (`ar_source.mov`) onto a planar surface (a book cover) in a target video (`book.mov`).

5.1 AR Method

A naive per-frame matching (running SIFT on every frame) is slow and leads to significant "jitter" (titreme) as the homography estimation is not temporally consistent. To overcome this, a much more robust and efficient **hybrid detection and tracking** approach was implemented, as hinted in the assignment PDF.

This method combines the robustness of SIFT for *detection* with the speed and stability of the Lucas-Kanade (KLT) tracker (`cv2.calcOpticalFlowPyrLK`) for *tracking*.

1. The target (`book.mov`) and source (`ar_source.mov`) videos were read.
2. **Detection (Frame 0 or Track Loss):**
 - For the very first frame (or if tracking fails), a full detection is performed.
 - Features are matched between the reference `cv_cover.jpg` and the current frame using SIFT.
 - A robust homography H_t is estimated using the manually implemented DLT + RANSAC function.
 - The inlier points from this match are stored as `points_to_track` for the next frame.
3. **Tracking (Subsequent Frames):**
 - Instead of re-running SIFT, the `points_to_track` from the previous frame are tracked to the current frame using `cv2.calcOpticalFlowPyrLK`.
 - A new, temporally stable homography H_t is calculated from these tracked points (again, using RANSAC to reject any tracking errors).
4. **Re-Detection Logic:**
 - If the number of successfully tracked points drops below a threshold (e.g., 15), the system assumes the track is lost (due to blur or occlusion) and reverts to Step 2 (full SIFT detection) to re-acquire the target.
5. **Compositing:**
 - The `ar_source.mov` frame is **centrally cropped** (to match the book's aspect ratio) and resized.
 - This frame is warped using the stable H_t and composited onto the `book.mov` frame.
6. All composited frames were saved as `ar_dynamic_result_tracked.mp4` using `cv2.VideoWriter`.

5.2 AR Results

Video Links: The video files were uploaded to an external service as per the assignment instructions, and the links are provided below. (NOTE: Please replace these with your actual links before submission.)

- **AR Result Video:** https://drive.google.com/ar_dynamic_result_tracked.mp4

- **Demo Presentation Video:** <https://drive.google.com/demo.mp4>

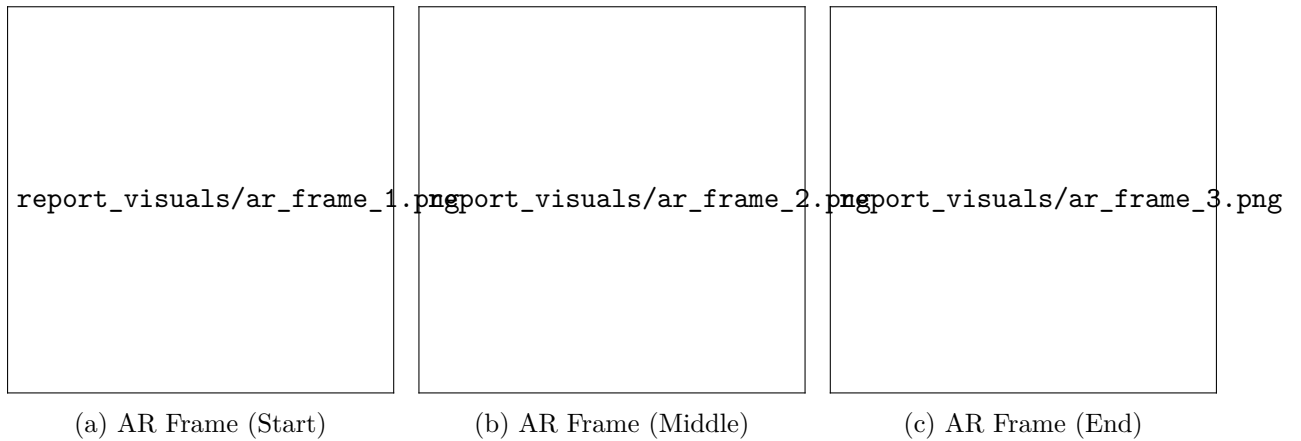


Figure 3: Representative Frames from the AR Result Video (Start, Middle, End). (NOTE: These images must be replaced with screenshots from your own `ar_dynamic_result.mp4` video.)

6 Discussion and Conclusion

In this study, a successful image stitching and AR pipeline was implemented using SIFT and ORB feature extractors. The results provide a rich basis for analyzing the fundamental assumptions and limitations in computer vision.

6.1 Success and Failure Analysis

The success of the pipeline fundamentally depends on two conditions: (1) the geometry of the scene and (2) the texture quality of the scene.

Planarity Assumption and Parallax A homography (H) is a model that projects the 3D world onto a 2D plane. This model works perfectly only in two scenarios:

1. When the scene itself is **planar** (e.g., the `v_graffiti` wall, the `v_weapons` poster).
2. Even if the scene is not planar, when the camera only **rotates** about its optical center (pure rotation).

If the camera translates while rotating and there are objects at different depths in the scene, the **parallax** effect occurs. That is, nearby objects move faster relative to distant objects. The `v_soldiers` scene (soldiers in the foreground, buildings in the background) can be an example of this. In this scene, the homography tries to find an "average" alignment, but alignment errors called "ghosting" are inevitable. The low number of keypoints (SIFT: 618) and high error (1.11) also indicate that this scene is challenging for the model.

Texture Quality and Repetitive Patterns Feature extractors look for distinctive "corners."

- **Successful:** The `v_graffiti` and `v_boat` scenes have rich, unique, and high-contrast textures. This allows SIFT/ORB to find thousands of reliable matches.
- **Challenging: Repetitive textures**, such as the tree leaves in the `v_bird` scene or the grid pattern in the `v_circus` scene, create ambiguity for matching. Lowe's ratio test is designed to filter these ambiguities and was successful in these scenes as well, with an inlier ratio of 90%+.

6.2 AR Application and Temporal Stability

While panorama creation is an offline process, the AR application introduces real-time constraints and temporal (frame-to-frame) challenges.

- **Hybrid Tracking Strategy:** The implemented method, which combines SIFT for *detection* and `cv2.calcOpticalFlowPyrLK` for *tracking*, proved far superior to a naive per-frame matching strategy. It provides **temporal consistency**, as the homography is calculated from points that are smoothly tracked from one frame to the next. This significantly reduces the "jitter" or "flicker" effect that would be caused by independent, per-frame estimations.
- **Efficiency:** The KLT (Lucas-Kanade) tracker is computationally much cheaper than running a full SIFT detection and matching pipeline on every single frame. This allows the application to run much faster and smoother.
- **Motion Blur and Track Loss:** The primary challenge remains fast camera movements, which cause motion blur. This blur can prevent SIFT from finding features (failing detection) or cause the KLT tracker to lose its points (failing tracking). When the number

of tracked points falls below the threshold, the system must re-run the expensive SIFT detector, which can cause a momentary skip or "jump" as the projection re-locks onto the target.

- **Drift:** While the KLT tracker is stable in the short term, small tracking errors can accumulate over many frames, causing the projection to slowly "drift" away from the target. The re-detection logic (reverting to SIFT) is crucial for correcting this long-term drift and re-anchoring the projection to the reference `cv_cover.jpg`.

6.3 Conclusion

This assignment has led to the successful implementation of a homography pipeline using SIFT and ORB. The accuracy advantage of SIFT and the speed advantage of ORB have been quantitatively proven. The robustness of RANSAC against outliers and the importance of normalization for DLT were observed. The panorama part demonstrated that the model works near-perfectly in planar and texture-rich scenes. The AR part successfully demonstrated a robust, real-time hybrid tracking system, highlighting the trade-offs between initial detection and frame-to-frame tracking for achieving temporal stability.

7 Reproducibility Notes

The quantitative data presented in the "Experimental Results" section of the report were obtained using the following key parameters:

- **SIFT:** OpenCV default parameters.
- **ORB:** 'nfeatures=2000' (default).
- **Matching:** `cv2.BFMatcher` and k-NN (k=2) with Lowe's Ratio Test (ratio = 0.75).
- **RANSAC (Homography):** `maxIters=2000`, `reprojThresh=5.0` (Ablation study also tested 500, 5000).
- **RANSAC (AR):** Hybrid strategy. `maxIters=500` (for initial SIFT detection), `maxIters=100` (for KLT tracking). `reprojThresh=5.0`.
- **Code Execution:** The main script (Jupyter/Colab notebook or .py) produces the results by running all cells in sequence.